```
function [numNormalTubes, numShapeAnomalies, totalTubesInImage, shapeMask, annotatedImage] =
shapeAnomalyDetection(img, colorStats, numColorAnomalies)

gs = im2gray(img);

    % Apply a medium filter to the image using a 15 pixel x 15 pixel size
    % square to remove texture of the green background
    % (i.e. to remove faded/scraped parts of the assembly line belt)
    gsSmooth = medfilt2(gs, [15 15]);

    % Apply adaptive thresholding to the previously medium-filtered image
    % This calculates a threshold based on the local mean intensity of the
    % vicinity of each pixel that is evaluated.
    % This actively searches for "bright", shiny surfaces (like the tubes)
    % against the background.
    % The resulting threshold is then used with imbinarize() to create a binary
    % image
    T = adaptthresh(gsSmooth, 0.45, 'ForegroundPolarity', 'bright');
    shapeMask = imbinarize(gsSmooth, T);

    % Fill in "hole" regions within the tubes to make them appear solid on the
    % mask using imfill()
    % Remove small objects from the mask that are smaller than 500 pixels using
    % bwareaopen()
    shapeMask = imfill(shapeMask, 'holes');
    shapeMask = bwareaopen(shapeMask, 500);

    % Create a disk structural image with radius = 6 pixels
    % to create an open image with the mask. This serves to address cases where
    % tubes are falsely classified as one tube due to their close proximity.
    breakSE = strel('disk', 6);
    shapeMask = imopen(shapeMask, breakSE);

    % Sets the corners of the image to 0 to prevent noise from affecting the
    % image analysis
    cornerSize = 50;
    shapeMask(1:cornerSize, 1:cornerSize) = 0;
    shapeMask(1:cornerSize, end-cornerSize:end) = 0;
    shapeMask(end-cornerSize:end, 1:cornerSize) = 0;
    shapeMask(end-cornerSize:end, end-cornerSize:end) = 0;

    % Measure the shiny silver tubes based on the shape mask using
    % regionprops()
    % ***IMPORTANT***
    %***************
    % regionprops() objects serve as a structure that tracks several properties
    % of images. In this case we are tracking:
    % 1) Centroid --> To determine the center of each tube
    %
    % 2) MajorAxisLength --> To determine the length of each tube
    %
    % 3) MinorAxisLength --> To determine the width of each tube
    %
    % 4) Orientation --> To determine the angle at which each tube is oriented.
    %                    This is for tightly fitting a rectangle to each tube
    %                    in order to classify each object after processing is
    %                    complete. Otherwise, the rectangles would be misplaced
    %                    and not line up with each tube correctly.
    %
    % 5) Area --> To track how much Area each tube takes up in order to
    %             determine a mean Area, to which other objects in the image
```

```
61      %             are compared against. Objects significantly smaller or larger than this
62      %             mean area are classified as anomalies.
63      %
64      % 6) Eccentricity --> To  determine how "elongated" an object is. This
65      %                     address cases where the tube is cut off (i.e. similar width but much
    shorter), and hence
66      %                     significantly shorter than a standard size tube.
67      %
68      % 7) Perimeter --> Returns the exact Perimeter of a region
69      %
70      shapeStats = regionprops(shapeMask, 'Centroid', 'MajorAxisLength', 'MinorAxisLength',
    'Orientation', 'Area', 'Eccentricity', 'Perimeter');
71
72      % Number each entry
73      for k = 1:length(shapeStats)
74          shapeStats(k).Num = k;
75      end
76
77      % Assign significance to be false by default
78      for k = 1:length(shapeStats)
79          shapeStats(k).significant = false;
80      end
81
82      % Difference between rectangular perimeter and calculated perimeter,
83      % Used to detect abnormalities such as dents which significantly
84      % decreases this value
85      for k = 1:length(shapeStats)
86          shapeStats(k).PerimeterDifference = (2*shapeStats(k).MajorAxisLength +
    2*shapeStats(k).MinorAxisLength) - shapeStats(k).Perimeter;
87      end
88
89      % ========================================================================
90      % METRICS TRACKING & VISUALIZATION
91      % ========================================================================
92      hiddenFig = figure("Visible","off");
93      imshow(img);
94      hold on;
95
96      % Initialize counters for the silver tubes
97      numShapeAnomalies = 0;
98      numNormalTubes = 0;
99
100     % Calibration variables for average tube area
101     % You must adjust these three numbers based on a "Good" image!
102     minNormalArea = 2000;         % Anything smaller than this is ignored as noise
103     maxNormalArea = 5200;       % Anything larger than this is flagged as crushed
104     minNormalEccentricity = 0.950; % Anything less elongated than this is flagged
105     maxNormalEccentricity = 0.999; % Anything more elongated than this is flagged
106     minPerimeterDifference = 35; % Anything less than this is flagged (used to detect dents)
107     minDoublePerimeterDifference = 100;
108     maxTubeLen = 200;
109     minTubeLen = 110;
110
111     % Draw and Log Color Anomalies onto original image
112     for c = 1:numColorAnomalies
113         % The below statements will plot red boxes over the tubes flagged as a
114         % color anomaly.
115         box = colorStats(c).BoundingBox;
116         plot([box(1), box(1)+box(3), box(1)+box(3), box(1), box(1)], ...
117              [box(2), box(2), box(2)+box(4), box(2)+box(4), box(2)], 'r-', 'LineWidth', 2);
118     end
```

```
119
120     % Draw and Log Shape Anomalies vs. Normal Tubes
121     for k = 1:length(shapeStats)
122
123         thisArea = shapeStats(k).Area;
124
125         % Apply a noise filter using "historical" tube statistics that are set
126         % as constants in this script (adjust as deemed necessary to fine tune
127         % this part of the code)
128         if thisArea < minNormalArea
129             continue;
130         end
131
132         % labels this ROI as significant (used for data analysis)
133         shapeStats(k).significant = true;
134
135         thisEcc = shapeStats(k).Eccentricity;
136
137         % Code to tightly fit a rectangle around each tube.
138         center = shapeStats(k).Centroid;
139         majorLen = shapeStats(k).MajorAxisLength;
140         minorLen = shapeStats(k).MinorAxisLength;
141         angle = -shapeStats(k).Orientation;
142         perimeterDifference = shapeStats(k).PerimeterDifference;
143
144         L = majorLen / 2; W = minorLen / 2;
145         x = [-L,  L,  L, -L, -L]; y = [-W, -W,  W,  W, -W];
146
147         theta = deg2rad(angle);
148         R = [cos(theta), -sin(theta); sin(theta),  cos(theta)];
149         rotCoords = R * [x; y];
150
151         boxX = rotCoords(1, :) + center(1);
152         boxY = rotCoords(2, :) + center(2);
153
154
155         % Classify and count the number of normal and abnormal tubes
156         if ((((thisArea <= maxNormalArea) && (thisArea >= minNormalArea))... %Checks if the
    signular tube matches length requirements
157             && ((thisEcc >= minNormalEccentricity) && (thisEcc <= maxNormalEccentricity))... %
    Checks to see if the tube aligns with Eccentricity min & max
158             && (perimeterDifference >= minPerimeterDifference)...%Checks if Perimeter difference
    is large enough for a singular tube
159             && (majorLen < maxTubeLen) && (majorLen > minTubeLen))... % Checks if tube meets
    length requirements to be classified as a singular tube
160             || (majorLen >= maxTubeLen) && (perimeterDifference >=
    minDoublePerimeterDifference)); % Detects if a tube is "merged" with another and judges only
    its combined PD
161
162
163             plot(boxX, boxY, 'g-', 'LineWidth', 2); % Flag as Normal
164
165             numNormalTubes = numNormalTubes + 1;
166         else
167
168             plot(boxX, boxY, 'y-', 'LineWidth', 2); % Flag as Anomaly
169
170             numShapeAnomalies = numShapeAnomalies + 1;
171         end
172
173         % Label each with its corresponding number for easier data lookup
```

```
174         % from the output table
175
176         text(center(1), center(2), string(shapeStats(k).Num), "FontSize", 14, "Color", 'black');
177
178     end
179
180     % Turn off hold after all bounding boxes are drawn
181     % onto the original image
182     hold off;
183
184     % Take a screenshot of the current plot/axis w/ gca (i.e. bounding boxes atop the
185     % original image) to save the annotated picture for use in the "fancier" program output
186     frame = getframe(gca);
187     annotatedImage = frame.cdata;
188     close(hiddenFig);
189
190     % Print some results to the Command Window for troubleshooting purposes
191     totalTubesInImage = numNormalTubes + numShapeAnomalies + numColorAnomalies;
192
193 end
```